

Next.js と Django を用いた在庫管理システムの設計仕様

k24015 石丸凜弥¹

概要：本稿では、Next.js と Django を組み合わせた Web ベース在庫管理システムの設計仕様を報告する。本システムは、商品の登録・在庫数の管理・売上の一括登録・月別集計をブラウザ上で実現するものである。設計において特に考慮した点は、JWT を httpOnly Cookie で管理することによるセキュアな認証設計、自己参照外部キーを用いたカテゴリ階層の柔軟な表現、および CSV 一括登録における同期・非同期 2 方式の使い分けである。本稿ではシステム要件、機能設計、データベース設計、API 設計、実装の各観点からこれらの設計判断とその根拠を述べる。

Design of an Inventory Management System Using Next.js and Django

k24015 ISHIMARU RINYA¹

1. 序論

本稿で報告するシステムは、フロントエンドに Next.js、バックエンドに Django を採用した SPA (Single Page Application) 構成の在庫管理 Web アプリケーションである。設計上の主な課題として、(1) ステートレスな REST API に適した認証方式の選択、(2) 将来的な拡張を考慮したカテゴリ階層の表現、(3) 大量データを対象とした売上一括登録での処理方式の選択、の 3 点があった。本稿では、これら 3 点の設計判断とその根拠を中心に、システム全体の設計仕様を述べる。

2. システム要件

本システムは商品の在庫管理を目的とした Web ベースのシステムである。ユーザが商品の登録・在庫の更新・売上の集計を行える。認証機能は JWT を用いた設計を行ったが、フロントエンドとの連携に未解決の不具合があるため、現状は無効化して運用している (8 節参照)。主要な機能要件を表 1 にまとめる。

在庫整合性の観点から、卸し数量は現在の在庫数 (仕入れ合計 - 売上合計) を超えられないという制約をアプリケーション層で保証する。また、セキュリティ上の要件と

表 1 主要機能要件

機能	概要
認証機能	JWT 認証によるログイン・ログアウト、15 分でのトークン自動期限切れ
商品管理 在庫管理	商品の登録・一覧表示・編集・削除 (CRUD) 仕入れ (在庫増)・卸し (在庫減) の登録と履歴確認
CSV 一括登録	売上データの CSV ファイルによる同期・非同期一括登録
売上集計	月別売上数の集計と一覧表示

して、JWT アクセストークンの有効期限を 15 分に設定し、不正アクセスのリスクを低減する設計とした [1]。

3. 機能設計

3.1 認証機能

認証には JWT[2] を用いた設計を採用した。REST API との組み合わせで一般的に使われるセッション方式と比較すると、JWT はサーバ側にセッション情報を保持しないステートレスな設計であるため、水平スケールアウト時にセッション共有の仕組みを別途用意する必要がない点が利点である。

JWT の保管場所として、本設計では httpOnly Cookie を採用した。LocalStorage は JavaScript から自由に読み取れるため XSS 攻撃によるトークン漏洩リスクがある [3] のに

¹ 愛知工業大学 情報科学部
Department of Information Science, Aichi Institute of Technology

対し、httpOnly Cookie はブラウザがリクエスト時に自動付与し、JavaScript からアクセスできないため XSS 耐性が高い。アクセストークンの有効期限は 15 分、Cookie の保持期間は 12 時間を設計値とした。

ただし、本実装ではフロントエンドとバックエンド間の認証フローに未解決の不具合が存在し、ログイン後の API 呼び出しが正常に通過できない状態である。そのため現状の動作確認環境では、REST_FRAMEWORK の DEFAULT_AUTHENTICATION_CLASSES を空タプル、DEFAULT_PERMISSION_CLASSES を空リストに設定して認証を無効化している。認証機能の完全な実装は今後の課題である（8 節参照）。

3.2 商品管理機能

商品情報の登録・一覧・編集・削除（CRUD）を提供する。商品には名称・単価・説明・カテゴリを設定でき、商品 ID は自動採番とする。商品名と単価は必須項目とし、単価は 1~99,999,999 円の範囲での入力を求める。入力値の検証はフロントエンドの react-hook-form[4] によるフォームバリデーションと、バックエンドの DRF シリアライザによるバリデーションの 2 層で行う。

3.3 在庫管理機能

商品ごとに仕入れ（在庫増加）と卸し（在庫減少）を記録する。卸し登録時には、現在の在庫数（仕入れ合計 - 売上合計）が卸し要求数量を下回る場合に操作を拒否し、在庫数がゼロを下回ることを防止する制約をアプリケーション層で実装する（詳細は 7.4 節）。

3.4 CSV 一括売上登録機能

大量の売上データを CSV ファイルで一括登録する機能である。CSV には商品 ID(product)、日付(date)、数量(quantity) の 3 フィールドが必要である。同期処理と非同期処理の 2 方式を提供しており、データ量や用途に応じてユーザーが選択できる設計とした（詳細は 7.5 節）。

3.5 売上集計機能

登録された売上データを月単位で集計し、月別の売上合計数量を一覧表示する。集計は Django ORM の TruncMonth と Sum を組み合わせて SQL 集計として実装し、アプリケーション側での集計処理を排除することでパフォーマンスを確保している。

4. 画面設計

本システムの画面構成を表 2 に示す。

ログイン画面を除く各画面は、ヘッダー・サイドバー・フッターで構成される共通レイアウトを持ち、Next.js の layout.tsx[5] によって実現している。サイドバーには商

表 2 画面構成と主要機能

画面名	主要機能
ログイン画面	認証, 入力バリデーション
商品一覧画面	商品一覧, 追加・編集・削除
商品在庫管理画面	仕入れ・卸し操作, 在庫履歴一覧
売上一括登録画面	CSV アップロード, 月別売上集計表示

品一覧画面と売上一括登録画面へのナビゲーションリンクを配置した。

画面間の遷移は図 1 に示す通りである。設計上は、未認証状態でのアクセスはログイン画面にリダイレクトし、axios のレスポンスインターセプター [6] で 401 エラーを検知した際にリフレッシュトークンによる再認証を行う設計とした。ただし、認証機能は現状無効化されているため、これらの処理は動作していない。

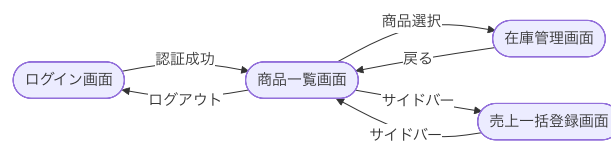


図 1 画面遷移図

商品一覧画面では、Material UI[7] の TableContainer で商品一覧を表示し、編集ボタンのクリックによってインライン編集エリアを展開する設計とした。React の useState フックで「現在編集集中の商品 ID」を管理することで、同時に 1 件だけ編集状態にある制御を実装している。

在庫管理画面では、上部の操作フォームから仕入れ・卸しを行い、画面下部に在庫履歴を最新順で一覧表示する構成とした。在庫履歴は仕入れレコードと売上レコードをバックエンドで UNION 結合し、処理種別（仕入れ/卸し）・日時・数量を統一フォーマットで返す設計である（詳細は 6.3 節）。

5. データベース設計

5.1 テーブル構成と ER 概要

本システムのデータベースには MySQL 8[8] を採用した。リレーショナルデータベースを選択した理由は、在庫数の計算が複数テーブルの集計クエリに依存しており、JOIN・UNION・GROUP BY といった SQL 操作の表現力が必要であることに加え、トランザクション保証（ACID）が在庫整合性の維持に不可欠であるためである。

テーブル構成は category・product・purchase・sales・sales_file の 5 テーブルであり、以下の各節でスキーマと設計根拠を述べる。

5.2 カテゴリ・商品テーブル

カテゴリテーブルのスキーマを表 3 に、商品テーブルのスキーマを表 4 に示す。parent_category_id は同テーブルの id を参照する自己参照外部キーであり、NULL の場

合はルートカテゴリを意味する。category.id はカテゴリテーブルの外部キーであり、カテゴリ未設定の商品を許容するため NULL を許可する。

表 3 カテゴリテーブルのスキーマ

カラム名	型	制約
id	bigint	PK, AUTO_INCREMENT
name	varchar(100)	NOT NULL
parent_category_id	bigint	FK(category.id), NULL 可

表 4 商品テーブルのスキーマ

カラム名	型	制約
id	bigint	PK, AUTO_INCREMENT
name	varchar(100)	NOT NULL
price	int	NOT NULL
category_id	bigint	FK(category.id), NULL 可
description	longtext	NULL 可

カテゴリ階層の実装には、隣接リスト (adjacency list) モデルを採用した。隣接リストは各行が親カテゴリへの参照のみを保持するシンプルな構造であり、階層の深さを事前に定義する必要がないため、「家電 > スマートフォン > Android 端末」のように後から深さを増やす変更にも対応できる。他の候補として固定深さのテーブル分割 (category_l1, category_l2 等) も考えられるが、階層追加のたびにスキーマ変更が必要となり拡張性に欠ける。

5.3 仕入れ・売上テーブル

仕入れテーブルのスキーマを表 5 に、売上テーブルのスキーマを表 6 に示す。売上テーブルには import_file_id フィールドを設け、どの CSV ファイルから登録されたデータかを追跡可能にした。これにより、特定ファイル分の売上データを集計・削除する操作が容易になる。

表 5 仕入れテーブルのスキーマ

カラム名	型	制約
id	bigint	PK, AUTO_INCREMENT
product_id	bigint	FK(product.id), NOT NULL
quantity	int	NOT NULL
purchase_date	datetime	NOT NULL

表 6 売上テーブルのスキーマ

カラム名	型	制約
id	bigint	PK, AUTO_INCREMENT
product_id	bigint	FK(product.id), NOT NULL
quantity	int	NOT NULL
sales_date	datetime	NOT NULL
import_file_id	bigint	FK(sales_file.id), NOT NULL

5.4 売上ファイルテーブルとステータス設計

売上ファイルテーブルのスキーマと status カラムの値を表 7 に示す。

表 7 売上ファイルテーブルのスキーマ

カラム名	型	制約
id	bigint	PK, AUTO_INCREMENT
file_name	varchar(100)	NOT NULL
status	int	NOT NULL

表 8 status カラムの値

値	状態
0	同期処理完了
1	非同期 (未処理)
2	非同期 (処理済)

このステータス設計は、非同期処理の進捗を DB で管理するために設けた。バッチ処理コマンドは status=1 のレコードのみを処理対象とし、処理完了後に status=2 に更新する。なお、現在の設計では「処理中」状態が存在しないため、バッチプロセスの二重起動による同一レコードの重複処理リスクがある。本実装では後述の select_for_update によって部分的に対処しているが、完全な解決には「処理中 (status=3)」状態の追加が望ましい課題として残る。

6. API 設計

各 API エンドポイントの一覧を表 9 に示す。すべてのエンドポイントはベースパス /api/inventory/ 以下に配置する。ログイン・ログアウト・リトライを除くエンドポイントは JWT 認証が必須である。

表 9 API エンドポイント一覧

パス	メソッド	機能
login/	POST	JWT 発行・Cookie 設定
logout/	POST	Cookie 削除
retry/	POST	トークン再発行
products/	GET/POST	商品一覧取得・登録
products/<id>/	PUT/DELETE	商品更新・削除
inventories/<id>/	GET	在庫履歴取得
purchases/	POST	仕入れ登録
sales/	POST	売上登録
sync/	POST	CSV 同期登録
async/	POST	CSV 非同期登録
summary/	GET	月別売上集計

6.1 認証 API の設計

認証 API はコードとして実装されているが、前述の通り現在は無効化されている。以下は設計意図の記録である。

ログイン API はリクエストボディのユーザ名・パスワードを `TokenObtainPairSerializer` で検証し、アクセストークンとリフレッシュトークンの両方を `httpOnly` Cookie にセットして応答する設計である [9]。一般的な JWT 実装が `Authorization` ヘッダを使うのに対し、`AccessJWTAuthentication` クラスをカスタム実装して Cookie からトークンを HTTP ヘッダへ移し替える処理を加えることで、フロントエンド側のトークン管理コードを不要にし、XSS 耐性を高める意図がある [3]。

リトライ API は期限切れのアクセストークンをリフレッシュトークンで更新し、新しいトークンペアを Cookie に保存して応答する設計である。図 2 に設計上の認証フローを示す。

なお、ログアウト API はバックエンドで Cookie を削除する実装となっているが、現在のフロントエンド実装 (`layout.tsx`) ではログアウトボタンクリック時に API を呼び出さず、`router.replace("/login")` によるページ遷移のみを行っている。そのため現状ではログアウト後も Cookie が残存する状態であり、ログアウト API の呼び出しを追加することが今後の課題である。

6.2 商品 API の設計

商品 API は `/api/inventory/products/` に GET・POST・PUT・DELETE を実装し、Django の `APIView` を継承した `ProductView` クラスとして定義した。本システムでは標準的な CRUD 操作を `ModelViewSet` [10] で自動生成する実装も合わせて用意しているが、バリデーションやエラーレスポンスの細かな制御が必要な場合に `APIView` による手動実装が有効であると考え。

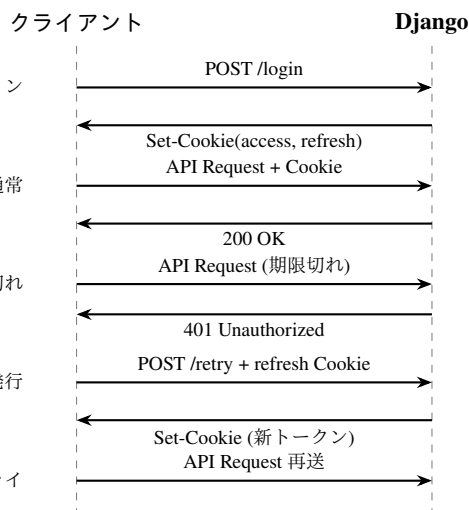


図 2 JWT 認証フロー

6.3 在庫履歴 API の設計

在庫履歴 API は、仕入れテーブルと売上テーブルのレコードを統合して返す。2つのテーブルを統合するため、ORM の UNION 操作を活用した設計とした。仕入れクエリには `type=Value('1')`、売上クエリには `type=Value('2')` を付与することでレコードの種別を識別可能にし、日時順に並べて応答する。この設計によりフロントエンドは単一の API からすべての在庫変動履歴を取得でき、表示ロジックをシンプルに保てる。

6.4 仕入れ・売上 API の設計

仕入れ登録 API は商品ごとの仕入れ数を登録するためのエンドポイントである。エンドポイントは `/api/inventory/purchases/` であり、POST メソッドのみを受け取る。リクエストボディには商品 ID・数量・仕入れ日時を含む JSON オブジェクトを送信する。`PurchaseSerializer` を使用してリクエストデータを検証し、検証に成功した場合は仕入れ情報を登録して HTTP ステータスコード 201 Created を返す。

売上登録 API は商品ごとの卸し数を登録するためのエンドポイントである。エンドポイントは `/api/inventory/sales/` であり、POST メソッドのみを受け取る。登録時には在庫整合性チェックを行い、卸し数量が現在の在庫数を超える場合は `BusinessException` でエラーを返す (詳細は 7.4 節)。検証に成功した場合は HTTP ステータスコード 201 Created を返す。

6.5 CSV 一括登録 API の設計

CSV 一括登録 API は CSV ファイルをアップロードして売上データを一括登録するためのエンドポイントである。同期処理のエンドポイントは `/api/inventory/sync/`、非同期処理のエンドポイントは `/api/inventory/async/` であり、いずれも POST メソッドのみを受け取る。リクエス

トボディには CSV ファイルを含むマルチパートフォームデータを送信する。CSV ファイルには `product · date · quantity` の 3 フィールドが必要である。2 方式の設計詳細は 7.5 節で述べる。

6.6 売上集計 API の設計

売上集計 API は月別の売上数を集計して返すためのエンドポイントである。エンドポイントは `/api/inventory/summary/` であり、GET メソッドのみを受け取る。Django ORM の `TruncMonth` と `Sum` を組み合わせ、月別に集計した売上数を JSON 形式でレスポンスとして返す。リクエストボディは不要であり、集計結果は年月と売上数量のペアで返す。

7. 実装

7.1 実装環境

本システムの実装環境を表 10 に示す。フロントエンドに Next.js[11]、バックエンドに Django[15] を採用した。Next.js は React[12] ベースのフレームワークであり、サーバサイドレンダリング (SSR) とクライアントサイドレンダリング (CSR) をページ・コンポーネント単位で使い分けられる点が特徴である。Django は Python ベースの Web フレームワークであり、売上 CSV 処理に必要な pandas ライブラリとの親和性の高さが採用理由の一つである。

表 10 実装環境

項目	バージョン
Node.js	18
Next.js	13.5.4
React	18.2.0
Emotion	11.11.1
Material UI	5.14.13
TypeScript	5.2.2
Django	4.2.6
Python	3.11
MySQL	8

7.2 環境構築

開発環境は VS Code Dev Container[13] と Docker Compose[14] を組み合わせて構築した。Docker Compose はデータベース (MySQL) コンテナのみを管理し、フロントエンドとバックエンドはそれぞれ独立した Dev Container 上で実行する構成とした。

この構成を選んだ理由は、フロントエンド・バックエンドのホットリロードを Docker 内で実行するよりも Dev Container (ホストマシン寄りの環境) で行う方がファイル変更の検知が速く、開発時の体験が向上するためである。一方、DB はデータの永続化が必要なためコンテナ化が適切と判断した。

データベースコンテナは docker image `mysql:8` を使用し、ポート 53306 で公開する。環境変数 `MYSQL_ROOT_PASSWORD` でルートパスワードを、`TZ` でタイムゾーンを `Asia/Tokyo` に設定する。

バックエンドの Dev Container は Microsoft が提供する Dev Container 用イメージ (`mcr.microsoft.com/devcontainers/python:1-3.11-bullseye`) を使用する。Django のプロジェクト構成は、`settings.py` を環境別 (`base.py · development.py` 等) に分割する構成とした。環境間で共通の設定は `base.py` に記述し、開発環境固有の DB 接続情報等は `development.py` で上書きする。サーバ起動時に `--settings config.settings.development` を指定することで開発用設定を読み込む。

フロントエンドの Dev Container は Docker image `node:18` を使用する。バックエンドへの API リクエストは、Next.js の Rewrites 機能 [16] を用いて `/api/:path*` を `http://host.docker.internal:8000/api/:path*` に転送する設定を行った。この構成によりフロントエンドとバックエンドのオリジンが同一になるため、CORS エラーを回避できる。

7.3 モデル実装

Django のモデルは Python クラスとして定義し、ORM 経由で DB を操作する設計とした [15]。フィールドの型と Django ORM フィールドの対応を表 11 に示す。

表 11 データ型と Django ORM フィールドの対応

データ型	フィールドクラス
int	IntegerField
bigint	BigIntegerField
varchar	CharField
longtext	TextField
datetime	DateTimeField
FK	ForeignKey

売上ファイルのステータスには `IntegerChoices` を用いて定数を定義した。これにより、ソースコード中でマジックナンバーを使わず `Status.SYNC · Status.ASYNC_UNPROCESSED · Status.ASYNC_PROCESSED` として参照でき、可読性と保守性が向上する。

7.4 在庫整合性チェックの実装

卸し登録時の在庫整合性チェックは、仕入れ合計と売上合計をそれぞれ SQL の SUM 集計で取得し、式 (1) を満たさない場合に `BusinessException` を送出して操作を拒否する実装とした。

$$\sum purchase.quantity \geq \sum sales.quantity + new_quantity \quad (1)$$

集計時には `Coalesce(Sum('quantity'), 0)` を使用する。 `Coalesce` で `NULL` を `0` に変換することで、仕入れや売上がまだ1件も存在しない商品に対しても `NullPointerException` なく正しく動作する。なお、現在の実装では複数の同時リクエストが競合した場合にチェックをすり抜ける可能性があり、今後の課題として残る（8節参照）。

7.5 CSV一括登録の実装

CSV ファイルの解析には `pandas` ライブラリの `read_csv` を使用した。同期処理と非同期処理の2方式における処理フローを図3に示す。

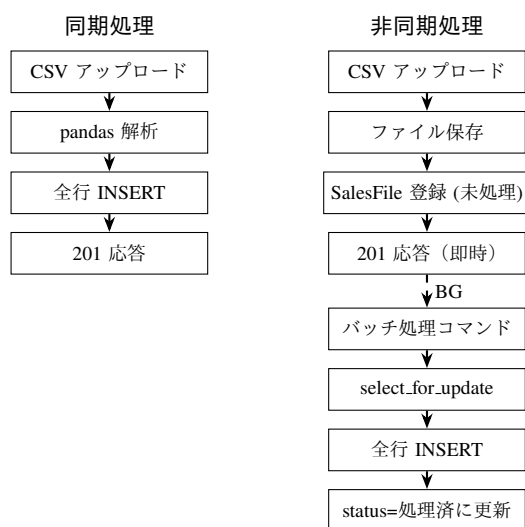


図3 同期・非同期 CSV 処理フロー比較

同期処理は、CSV のすべての行を `INSERT` し終わるまで `HTTP` レスポンスを返さない。データ量が少ない場合はシンプルで即座にフィードバックが得られるため有効だが、大量データでは `HTTP` タイムアウトが発生するリスクがある。

非同期処理では、CSV ファイルの保存と `SalesFile` レコードの作成のみを行い即座に `201` を応答する。実際の `INSERT` 処理は `Django` の `BaseCommand` を継承したバッチコマンドが担う。バッチコマンドは `status=ASYNC_UNPROCESSED` のレコードを順次処理し、完了後に `status=ASYNC_PROCESSED` へ更新する。この際、`select_for_update()` を用いて処理対象レコードに対して行レベルロックを取得することで、バッチプロセスが複数起動された場合の二重処理を防止している [8]。さらに `@transaction.atomic` デコレータによってトランザクション管理を行い、処理途中の障害による不整合を防ぐ設計とした。

2方式の特性比較を表12に示す。

表12 同期・非同期処理の特性比較

項目	同期	非同期
応答タイミング	処理完了後	即時
タイムアウトリスク	あり	なし
完了確認	レスポンスで確認	手動で確認必要
実装の複雑さ	低	高
適したデータ量	少量	大量

7.6 フロントエンド実装

7.6.1 ディレクトリ構成

フロントエンドは `Next.js` の `App Router` [5] を用いて実装した。 `App Router` ではファイルベースのルーティングを採用しており、 `app/` ディレクトリ以下のディレクトリ構造がそのまま `URL` パスになる。 `layout.tsx` ファイルにはその階層以下のページ共通レイアウトを定義でき、 `page.tsx` ファイルには各ページのコンテンツを定義する。 `[id]` のような動的ルートで商品 `ID` に応じた在庫管理ページを実現している。

7.6.2 フェッチャーの実装

`API` との通信は `plugins/axios.ts` に実装したフェッチャーを通じて行う。 `axios` [6] の `axios.create()` でインスタンスを作成し、 `Content-Type: application/json` をヘッダに設定して `JSON` 形式でリクエストを送信する。レスポンスインターセプターでは成功時にはそのままレスポンスを返し、 `401` エラー検知時はリトライ `API` を呼び出して新しいトークンを取得し元のリクエストを再送する設計とした。これにより、認証ロジックをアプリケーション全体の一箇所に集約できている。

7.6.3 商品一覧画面

商品一覧画面は `app/inventory/products/page.tsx` に実装する。 `useEffect` フックを使用して初期レンダリング時にフェッチャーで商品情報を取得し、 `Material UI` [7] の `TableContainer` コンポーネントで表形式に表示する。 `useState` フックで「現在編集中的商品 `ID`」を管理することで、同時に1件だけ編集状態にある制御を実現している。入力値の検証は `react-hook-form` [4] の `register` で行い、エラーメッセージをテキストエリア下部に表示する設計とした。

7.6.4 在庫管理画面

在庫管理画面は `app/inventory/products/[id]/page.tsx` に実装する。 `Next.js` の `App Router` では動的ルートのパラメータはコンポーネントの `params` プロパティとして渡されるため、 `params.id` で商品 `ID` を取得する。 `useEffect` フックで初期レンダリング時に商品情報と在庫履歴を取得する。画面上部の仕入れ・卸しフォームには数量入力欄と操作ボタンを配置し、 `react-hook-form` でバリデーションを行う。画面下部には在庫履歴をテーブル形式で最新順

に表示する。

7.6.5 売上一括登録画面

売上一括登録画面は `app/inventory/import_sales/page.tsx` [4] に実装する。 `useEffect` フックで月別売上集計を初期取得し、 `TableContainer` で表示する。 `Material UI` の `MuiFileInput` コンポーネントでファイル選択エリアを実装し、同期・非同期それぞれの登録ボタンと組み合わせてフォームを構成する。登録ボタンクリック時にフェッチャーで CSV 一括登録 API へリクエストを送信する設計とした。

8. まとめ

本稿では、Next.js と Django を用いた在庫管理 Web アプリケーションの設計仕様を報告した。序論で挙げた3点の設計課題に対する本システムの対応は以下の通りである。

認証設計として JWT を `httpOnly Cookie` で管理する方式を採用し、XSS に対して `LocalStorage` 保管より高い耐性を持つ設計とした。しかし現状ではフロントエンドとの認証連携に未解決の不具合があり、認証機能を無効化した状態で動作している。認証フローの不具合解消が最優先の課題であり、その後、発行済みトークンの即時無効化（トークンブラックリスト）への対応も必要となる。

自己参照外部キーによるカテゴリ階層は、固定深さのテーブル分割案と比較してシンプルな実装で可変深さを表現できた。ただし、循環参照（あるカテゴリが自身の子孫カテゴリを親に指定する）を防ぐ制約が現状では未実装であり、アプリケーション層での対策が必要である。

CSV 一括登録の同期・非同期2方式は、少量データから大量データまで対応できる柔軟な設計を実現した。 `select_for_update` を用いたロック取得で二重処理を防止しているが、処理完了をユーザに通知する仕組みが未実装であり、WebSocket または Server-Sent Events を用いた通知機構の追加が今後の課題である。

また、在庫整合性チェックはアプリケーション層のみで行っており、並行リクエストによるチェックのすり抜けリスクが残る。MySQL のトランザクション分離レベルとロック機構を活用した DB レベルでの保証がより堅牢な解決策となるが、本実装ではその対応は今後の課題として残した。

参考文献

- [1] OWASP Foundation, 「Authentication Cheat Sheet」, OWASP Cheat Sheet Series, (2026 年 6 月取得, https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html).
- [2] Jones, Michael B., Bradley, John, Sakimura, Nat, 「RFC 7519: JSON Web Token (JWT)」, Internet Engineering Task Force (IETF), (2026 年 6 月取得, <https://www.rfc-editor.org/rfc/rfc7519>).
- [3] OWASP Foundation, 「Session Management Cheat Sheet」,

OWASP Cheat Sheet Series, (2026 年 6 月取得, https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html).

Lau, Bill, 「React Hook Form」, React Hook Form, (2026 年 6 月取得, <https://react-hook-form.com/>).

- [5] Vercel, Inc., 「Next.js App Router: Getting Started」, Next.js Documentation, (2026 年 6 月取得, <https://nextjs.org/docs/app/getting-started>).

- [6] Axios contributors, 「Axios — Promise Based HTTP Client」, Axios, (2026 年 6 月取得, <https://axios.rest/pages/getting-started/first-steps>).

- [7] MUI SAS, 「Material UI — Overview」, Material UI, (2026 年 6 月取得, <https://mui.com/material-ui/getting-started/>).

- [8] Oracle Corporation, 「MySQL 8.0 Reference Manual」, MySQL, (2026 年 6 月取得, <https://dev.mysql.com/doc/refman/8.0/en/>).

- [9] Jazzband contributors, 「Simple JWT」, Simple JWT Documentation, (2026 年 6 月取得, <https://django-rest-framework-simplejwt.readthedocs.io/en/latest/>).

- [10] Encode OSS Ltd., 「Django REST Framework」, Django REST Framework, (2026 年 6 月取得, <https://www.django-rest-framework.org/>).

- [11] Vercel, Inc., 「Next.js Documentation」, Next.js, (2026 年 6 月取得, <https://nextjs.org/docs>).

- [12] Meta Platforms, Inc., 「React — The Library for Web and Native User Interfaces」, React, (2026 年 6 月取得, <https://react.dev/>).

- [13] Microsoft Corporation, 「Developing inside a Container — Visual Studio Code」, Visual Studio Code Documentation, (2026 年 6 月取得, <https://code.visualstudio.com/docs/devcontainers/containers>).

- [14] Docker, Inc., 「Docker Compose — Overview」, Docker Documentation, (2026 年 6 月取得, <https://docs.docker.com/compose/>).

- [15] Django Software Foundation, 「Django Documentation」, Django, (2026 年 6 月取得, <https://docs.djangoproject.com/en/4.2/>).

- [16] Vercel, Inc., 「next.config.js: rewrites」, Next.js Documentation, (2026 年 6 月取得, <https://nextjs.org/docs/app/api-reference/config/next-config-js/rewrites>).